Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase V · Alineamiento, aprendizaje por refuerzo, agentes e interpretabilidad

Módulo 23

Interpretabilidad y seguridad

Circuitos, superposición, atribución, evaluación y ataques adversarios

---

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 23: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
d, d_h, L	dimensión del flujo residual, dimensión de cada cabeza y número de capas
x_0, x_ℓ, x_{fin}	embedding, flujo residual tras la capa ℓ y flujo final
a_ℓ, m_ℓ	salidas de la atención y del MLP de la capa ℓ
$W_E \in \mathbb{R}^{d\times K}$, $W_U \in \mathbb{R}^{K\times d}$	matrices de embedding y de desembedding; $K = \lvert\mathcal{V}\rvert$
$W_{QK} = W_Q^\top W_K$, $W_{OV} = W_O W_V$	circuitos QK y OV de una cabeza
$\sigma(z) = 1/(1+e^{-z})$	función logística
$\text{IG}_i(x;x')$, ϕ_i	gradiente integrado y valor de Shapley de la entrada i
n, m, S, I_i	número de características, dimensión oculta, dispersión e importancia (modelo de juguete)
W_i, $\hat{W}_i = W_i/\lVert W_i\rVert$, D_i	dirección de la característica i y dimensión por característica
$f \in \mathbb{R}^F$, d_j, λ	código del autoencoder disperso, dirección del latente j y coeficiente de dispersión
pass@k, n, c	probabilidad de éxito con k intentos; muestras generadas y correctas por problema (tema 6)
conf_b, acc_b	confianza media y exactitud del intervalo b

Índice

Sobre este resumen **ii**

1 El flujo residual y la descomposición lineal **1**
1.1 El flujo residual como suma de aportaciones 1
1.2 Pesos virtuales y subespacios 2
1.3 Circuitos QK y OV 2
1.4 Transformers de cero y una capa 3
1.5 La *logit lens* y la normalización final 3

2 Sondas y métodos de atribución **5**
2.1 Sondas lineales 5
2.2 Qué demuestra una sonda 5
2.3 La sonda estructural 6
2.4 La hipótesis de representación lineal 6
2.5 Atribución por gradientes y gradientes integrados 7
2.6 Valores de Shapley y SHAP 8

3 Superposición **9**
3.1 Características, polisemanticidad y bases privilegiadas 9
3.2 Casi ortogonalidad en alta dimensión 9
3.3 El modelo de juguete 10
3.4 Dimensión por característica 11
3.5 Superposición como transición de fase 11

4 Autoencoders dispersos **12**
4.1 Aprendizaje de diccionarios 12
4.2 El autoencoder disperso 12
4.3 La penalización L^1: umbral suave y encogimiento 13
4.4 Métricas de un diccionario 13
4.5 Resultados publicados 14

5 Circuitos e intervenciones causales **15**

5.1 Composición de cabezas y cabezas de inducción 15
5.2 Intervenciones: ablación y parcheo de activaciones 16
5.3 Localización y edición de hechos: ROME 16
5.4 *Grokking* y el algoritmo de Fourier 17
5.5 Vectores de dirección 17
5.6 Trazado de circuitos en un modelo comercial 18

6 Evaluación, calibración y seguridad **19**

6.1 Conjuntos de evaluación y su incertidumbre 19
6.2 pass@k y su estimador insesgado 20
6.3 Calibración 20
6.4 Alucinaciones y reglas de puntuación 21
6.5 *Red teaming* y ataques adversarios 21
6.6 Estadística de los sucesos raros 22
6.7 Marcos de seguridad y conductas ocultas 22

Bibliografía **23**

Tema 1

El flujo residual y la descomposición lineal

Este tema expone la observación estructural en la que se basa casi toda la interpretabilidad mecanicista de los Transformers: cada componente lee del flujo residual mediante una proyección lineal y escribe en él sumando su salida, de modo que todo lo que se aplica linealmente al final se reparte sobre esa suma. La presentación sigue el marco matemático de Elhage *et al.* [14] (secciones «Transformer Overview», «Zero-Layer Transformers» y «One-Layer Attention-Only Transformers»), la vista del flujo residual del §7.2 de Jurafsky y Martin [25] y el §10.3 del mismo libro para la *logit lens*, propuesta por nostalgebraist [36] y corregida por Belrose *et al.* [4]. La arquitectura y la notación son las de los módulos 13 a 15.

1.1 El flujo residual como suma de aportaciones

Sea un Transformer decoder-only de L capas y dimensión d. Para simplificar, en este tema se prescinde de las normalizaciones (salvo en la apartado 1.5), como hacen Elhage *et al.* en su apartado «Model Simplifications» [14]; según ese trabajo, la normalización puede absorberse, salvo un factor escalar, en los pesos adyacentes.

Definición 1.1 (Flujo residual). Sea $x_0 \in \mathbb{R}^d$ el embedding del token (y de su posición) en una posición fija. Para $\ell = 1, \dots, L$, la capa ℓ calcula

$$x_{\ell-1/2} = x_{\ell-1} + a_\ell, \qquad x_\ell = x_{\ell-1/2} + m_\ell,$$

donde a_ℓ es la suma de las salidas de las cabezas de atención de la capa, que depende de los flujos $x_{\ell-1}$ de todas las posiciones anteriores, y m_ℓ es la salida del MLP, que depende de $x_{\ell-1/2}$ en la misma posición. La sucesión $x_0, x_{1/2}, x_1, \dots, x_L$ se llama *flujo residual* y $x_{\text{fin}} = x_L$ es el *flujo final*. Los logits del token siguiente son $z = W_U\, x_{\text{fin}} \in \mathbb{R}^K$.

Proposición 1.2 (Descomposición del flujo y de los logits). *Con la notación de la definición 1.1,*

$$x_{\text{fin}} = x_0 + \sum_{\ell=1}^{L} \big(a_\ell + m_\ell\big), \qquad z = W_U x_0 + \sum_{\ell=1}^{L} \big(W_U a_\ell + W_U m_\ell\big).$$

Definición 1.3 (Atribución directa a los logits). Sea c cualquiera de los sumandos x_0, a_ℓ, m_ℓ (o la salida de una sola cabeza, ya que a_ℓ es suma de las salidas de sus cabezas). Su *atribución directa a los logits* es el vector $W_U c \in \mathbb{R}^K$. Para comparar dos tokens t y t' se usa la *diferencia de logits* $(W_U c)_t - (W_U c)_{t'}$.

La palabra *directa* es importante. La salida a_1 influye en los logits por dos vías: directamente, a través del sumando $W_U a_1$, e indirectamente, porque modifica la entrada de todas las capas

posteriores y, con ello, $a_2, m_2, \ldots$. La proposición 1.2 solo reparte el primer tipo de efecto; el segundo exige intervenciones (tema 5).

1.2 Pesos virtuales y subespacios

Como el flujo residual no realiza ningún cálculo propio, dos componentes separados por varias capas se comunican a través de él como si estuvieran conectados directamente.

Definición 1.5 (Pesos virtuales). Sea $W_{\text{out}}^{(1)} \in \mathbb{R}^{d\times p}$ la matriz con la que un componente escribe en el flujo y $W_{\text{in}}^{(2)} \in \mathbb{R}^{q\times d}$ la matriz con la que un componente posterior lo lee. El producto $W_{\text{in}}^{(2)} W_{\text{out}}^{(1)} \in \mathbb{R}^{q\times p}$ se llama *matriz de pesos virtuales* entre ambos [14, «Virtual Weights and the Residual Stream as a Communication Channel»].

Por la proposición 1.2, la parte de la entrada del segundo componente que procede del primero es exactamente $W_{\text{in}}^{(2)} W_{\text{out}}^{(1)} h$, donde h es la activación interna del primero, sea cual sea el número de capas intermedias. Como cada cabeza escribe en un subespacio de dimensión a lo sumo $d_h \ll d$, componentes distintos pueden usar subespacios casi disjuntos del flujo; Elhage *et al.* describen el flujo como un canal de comunicación de ancho de banda limitado, en el que las dimensiones son un recurso escaso.

1.3 Circuitos QK y OV

Sea una cabeza de atención con matrices $W_Q, W_K, W_V \in \mathbb{R}^{d_h\times d}$ y $W_O \in \mathbb{R}^{d\times d_h}$ (módulo 13). En la posición i, con flujos $x_1, \ldots, x_i$ en las posiciones visibles, la cabeza calcula

$$s_{ij} = \frac{(W_Q x_i)^\top (W_K x_j)}{\sqrt{d_h}}, \qquad A_{ij} = \frac{e^{s_{ij}}}{\sum_{j'\le i} e^{s_{ij'}}}, \qquad h_i = W_O \sum_{j\le i} A_{ij}\, W_V x_j.$$

Definición 1.6 (Circuitos QK y OV). Se llaman *circuito QK* y *circuito OV* de la cabeza a las matrices $d \times d$

$$W_{QK} = W_Q^\top W_K, \qquad W_{OV} = W_O W_V.$$

Proposición 1.7 (Factorización de una cabeza). *Con la notación anterior:*

(i) las puntuaciones son la forma bilineal $s_{ij} = x_i^\top W_{QK}\, x_j/\sqrt{d_h}$, de modo que el patrón de atención solo depende de W_Q y W_K a través de W_{QK};

(ii) la salida es $h_i = \sum_{j\le i} A_{ij}\, W_{OV} x_j$, de modo que, fijado el patrón, la cabeza es lineal en los flujos de origen y solo depende de W_O y W_V a través de W_{OV};

(iii) $\operatorname{rango} W_{QK} \le d_h$ *y* $\operatorname{rango} W_{OV} \le d_h$.

El circuito QK decide *a dónde mira* la cabeza y el circuito OV, *qué copia* de la posición atendida y cómo lo escribe. Solo el patrón A es no lineal; Elhage *et al.* subrayan que, fijado A, la cabeza es una aplicación lineal [14, «Splitting Attention Head terms into Query-Key and Output-Value Circuits»]. Con $d = 512$ y $d_h = 64$, W_{OV} es una matriz de $512 \times 512 = 262\,144$ entradas, pero está determinada por los $2 \cdot 512 \cdot 64 = 65\,536$ parámetros de W_O y W_V.

1.4 Transformers de cero y una capa

Sea $W_E \in \mathbb{R}^{d\times K}$ la matriz de embedding, cuya columna t es el embedding del token t, y sea $e_t \in \mathbb{R}^K$ el vector canónico del token actual.

Proposición 1.8 (Transformer de cero capas). *Si $L = 0$ y se omiten los embeddings de posición, los logits son $z = W_U W_E\, e_t$, es decir, la columna t de la matriz $W_U W_E \in \mathbb{R}^{K\times K}$. La distribución del token siguiente depende solo del token actual: el modelo es un modelo de bigramas (módulo 1) con matriz de logits $W_U W_E$.*

Si se entrena con la entropía cruzada, la matriz $W_U W_E$, de rango a lo sumo d, aproxima los logaritmos de las probabilidades de bigramas del corpus [14, «Zero-Layer Transformers»].

Teorema 1.9 (Expansión en caminos de una capa de atención). *Sea un Transformer de una capa, solo de atención, con cabezas $h \in \mathcal{H}$, patrones A^h y circuitos W^h_{OV}, sin embeddings de posición y sin normalización. Sea t_j el token de la posición j. Los logits en la posición i son*

$$z_i = \underbrace{W_U W_E e_{t_i}}_{\text{camino directo}} + \sum_{h\in\mathcal{H}} \sum_{j\le i} A^h_{ij}\, \underbrace{W_U W^h_{OV} W_E}_{K\times K}\, e_{t_j},$$

y las puntuaciones de la cabeza h son $s^h_{ij} = e_{t_i}^\top \big(W_E^\top W^h_{QK} W_E\big) e_{t_j} / \sqrt{d_h}$.

El teorema muestra que un modelo de una capa es una suma de un término de bigramas y de un término por cabeza que relaciona tres tokens: el token de origen t_j (que el circuito OV copia o transforma), el de destino t_i (que, con t_j, determina la atención a través del circuito QK) y el token predicho. Elhage *et al.* llaman *skip-trigramas* a estas reglas de la forma «$A \dots B \to C$» [14, «Interpretation as Skip-Trigrams»]. Una cabeza cuyo circuito $W_U W_{OV} W_E$ tiene una diagonal grande y positiva *copia*: si atiende a una aparición previa del token A, aumenta el logit de A.

1.5 La *logit lens* y la normalización final

Definición 1.10 (*Logit lens*). Sea x_ℓ el flujo residual tras la capa ℓ (o tras cualquier subcapa). La *logit lens* es la distribución $\mathrm{softmax}(W_U x_\ell)$: la predicción que daría el modelo si las capas restantes no añadieran nada [36], [25, §10.3].

En modelos reales, la *logit lens* es poco fiable en las primeras capas: el flujo todavía no está expresado en la base que lee W_U. La *tuned lens* de Belrose *et al.* [4] sustituye $W_U x_\ell$ por $W_U(A_\ell x_\ell + b_\ell)$, con una transformación afín por capa entrenada para que su predicción se parezca a la del modelo completo.

Los modelos actuales aplican una normalización antes de W_U, de modo que $z = W_U\, \gamma \odot \mathrm{RMSNorm}(x_{\text{fin}})$, con $\mathrm{RMSNorm}(v) = v/\,\mathrm{RMS}(v)$ y $\mathrm{RMS}(v) = \big(\frac{1}{d}\sum_k v_k^2\big)^{1/2}$ (módulo 18).

Proposición 1.12 (La normalización rompe la aditividad). *Para $v \neq 0$ y $c \neq 0$, $\mathrm{RMSNorm}(cv) = \mathrm{signo}(c)\ \mathrm{RMSNorm}(v)$. En consecuencia,* RMSNorm *no es aditiva: en general $\mathrm{RMSNorm}(u+v) \neq \mathrm{RMSNorm}(u) + \mathrm{RMSNorm}(v)$. Sin embargo, si se fija $s = \mathrm{RMS}(x_{\text{fin}})$, se cumple*

$$z = \sum_c W_U\, \gamma \odot \frac{c}{s},$$

donde la suma recorre las aportaciones de la proposición 1.2.

La proposición justifica la práctica habitual de «congelar» la escala: se calcula s en la pasada hacia delante y se trata como una constante, con lo que la atribución directa vuelve a sumar exactamente los logits. El precio es que la atribución deja de reflejar que un componente que escribe mucho también reduce, a través de s, la contribución relativa de todos los demás.

Tema 2

Sondas y métodos de atribución

Este tema estudia dos familias de herramientas que interrogan a un modelo sin modificarlo. Las *sondas* preguntan si una propiedad está codificada en las activaciones de una capa; los *métodos de atribución* preguntan qué partes de la entrada son responsables de una salida. Las sondas lineales siguen a Alain y Bengio [1] y al §10.2 de Jurafsky y Martin [25]; las tareas de control, a Hewitt y Liang [22], y la sonda estructural, a Hewitt y Manning [23]. Los gradientes integrados y sus axiomas proceden de Sundararajan, Taly y Yan [42], y los valores de Shapley aplicados a la explicación de modelos, de Lundberg y Lee [32]. La regresión logística y su relación con los modelos generativos gaussianos remiten al módulo 8 y al §4.2 de Bishop [5].

2.1 Sondas lineales

Definición 2.1 (Sonda lineal). Sea $h_\ell(u) \in \mathbb{R}^d$ la activación de la capa ℓ de un modelo congelado para la entrada u, y sea $y(u) \in \{0, 1\}$ una propiedad de la entrada. Una *sonda lineal* es un clasificador logístico

$$p(y = 1 \mid h) = \sigma\big(w^\top h + w_0\big), \qquad \sigma(z) = \frac{1}{1 + e^{-z}},$$

entrenado con la entropía cruzada sobre pares $(h_\ell(u_i), y_i)$ sin propagar gradientes al modelo. Si la sonda alcanza una exactitud alta en datos de prueba, se dice que la propiedad es *linealmente decodificable* en la capa ℓ [1].

Alain y Bengio entrenaron sondas en cada capa de redes de visión y observaron que la separabilidad lineal de las clases aumenta de forma monótona con la profundidad, aunque el entrenamiento solo la exige en la última capa [1].

Proposición 2.2 (El entrenamiento de una sonda es convexo). *Sea* $\tilde{y}_i = 2y_i - 1 \in \{-1, 1\}$. *La pérdida media*

$$J(w, w_0) = \frac{1}{n} \sum_{i=1}^{n} \ln\big(1 + e^{-\tilde{y}_i (w^\top h_i + w_0)}\big)$$

coincide con la entropía cruzada de la sonda y es una función convexa de (w, w_0).

La convexidad garantiza que el resultado de la sonda no depende de la inicialización: si la sonda falla, no es por un mal mínimo local, sino porque la propiedad no es linealmente separable en esa capa (o porque faltan datos).

2.2 Qué demuestra una sonda

Que una sonda acierte no implica que el modelo use la propiedad. Pueden fallar dos cosas: que la sonda sea tan expresiva que aprenda la tarea por sí misma, y que la información esté

presente pero el modelo no la lea.

Definición 2.4 (Tarea de control y selectividad). Sea una tarea lingüística que asigna a cada aparición de una palabra una etiqueta de un conjunto $\mathcal{Y}$. Su *tarea de control* asigna a cada *tipo* de palabra una etiqueta de $\mathcal{Y}$ elegida al azar una vez y la usa para todas sus apariciones. La *selectividad* de una sonda es la diferencia entre su exactitud en la tarea lingüística y en la tarea de control [22, §2].

La tarea de control solo puede resolverse memorizando la correspondencia entre tipos de palabra y etiquetas, sin usar el contexto. Una sonda selectiva acierta la tarea real y falla la de control. Hewitt y Liang observan, para el etiquetado gramatical con la capa 1 de ELMo, que una sonda lineal alcanza el 97,2 % en la tarea y el 71,2 % en la de control, mientras que un perceptrón de una capa oculta alcanza el 97,3 % y el 92,8 % [22, §3.5]: la sonda más potente apenas gana exactitud y pierde casi toda la selectividad.

Observación 2.5. Decodificable no es lo mismo que usado. Para afirmar que el modelo *usa* una dirección hay que intervenir sobre ella y observar el cambio de conducta (tema 5); Jurafsky y Martin [25, §10.2] hacen la misma advertencia.

Una sonda tampoco identifica de forma única la dirección de un concepto. La proposición siguiente muestra que el vector w de la regresión logística no coincide en general con la diferencia de medias de las dos clases.

Proposición 2.6 (Clases gaussianas con covarianza común). *Sean* $h \mid y = k \sim \mathcal{N}(\mu_k, \Sigma)$ *para* $k \in \{0, 1\}$*, con* Σ *definida positiva y* $P(y = 1) = \pi$*. Entonces*

$$P(y = 1 \mid h) = \sigma\big(w^\top h + w_0\big), \qquad w = \Sigma^{-1}(\mu_1 - \mu_0).$$

2.3 La sonda estructural

Las sondas de clasificación preguntan por propiedades de una palabra. Hewitt y Manning preguntan por una estructura completa: el árbol de dependencias de la frase.

Definición 2.8 (Sonda estructural). Sean $h_1, \dots, h_T$ las representaciones de las palabras de una frase y $B \in \mathbb{R}^{r \times d}$. La *sonda estructural de distancia* define

$$d_B(h_i, h_j)^2 = \big(B(h_i - h_j)\big)^\top \big(B(h_i - h_j)\big) = \|B(h_i - h_j)\|^2$$

y entrena B para que $d_B(h_i, h_j)^2$ aproxime la distancia entre las palabras i y j en el árbol sintáctico. La *sonda de profundidad* entrena $\|Bh_i\|^2$ para que aproxime la profundidad de la palabra i en el árbol [23].

Como $d_B(h_i, h_j)^2 = (h_i - h_j)^\top B^\top B(h_i - h_j)$ y $B^\top B$ es semidefinida positiva, la sonda busca una métrica lineal en el espacio de representaciones. Hewitt y Manning encuentran transformaciones de este tipo para ELMo y BERT, y no para las representaciones de control, lo que sugiere que los árboles sintácticos están codificados en la geometría de esos modelos [23].

2.4 La hipótesis de representación lineal

Las sondas lineales, las analogías de palabras del módulo 2 (rey − hombre + mujer ≈ reina) y los vectores de dirección del tema 5 apuntan a una misma conjetura.

Definición 2.9 (Hipótesis de representación lineal). La *hipótesis de representación lineal* afirma que muchos conceptos de alto nivel están representados en el espacio de activaciones como *direcciones*: existe un vector v tal que la presencia o la intensidad del concepto se lee mediante la proyección $v^\top h$ y se modifica sumando múltiplos de v.

Park, Choe y Veitch [39] formalizan esta idea para los modelos de lenguaje y distinguen la versión «de lectura» (sondas) de la versión «de escritura» (intervenciones). El tema 3 explica por qué las direcciones, y no las neuronas individuales, son la unidad natural de análisis.

2.5 Atribución por gradientes y gradientes integrados

Sea $F : \mathbb{R}^n \to \mathbb{R}$ una función diferenciable a trozos, por ejemplo la probabilidad o el logit de un token en función de los embeddings de entrada. Un *método de atribución* asigna a cada entrada x un vector $(A_1(x), \dots, A_n(x))$ que mide la responsabilidad de cada coordenada en el valor $F(x)$. Muchos métodos lo hacen respecto de una *base* x' que representa la ausencia de información (por ejemplo, el embedding nulo).

Definición 2.10 (Axiomas de sensibilidad y completitud). Un método de atribución con base x' cumple:

(i) *sensibilidad* (a) si, para toda x que difiere de x' en una sola coordenada i con $F(x) \neq F(x')$, se tiene $A_i(x) \neq 0$;

(ii) *invariancia ante la implementación* si da las mismas atribuciones a dos redes que calculan la misma función;

(iii) *completitud* si $\sum_{i=1}^{n} A_i(x) = F(x) - F(x')$.

Definición 2.12 (Gradientes integrados). El *gradiente integrado* de la coordenada i en x, con base x', es

$$\mathrm{IG}_i(x; x') = (x_i - x'_i) \int_0^1 \frac{\partial F}{\partial x_i}\big(x' + \alpha(x - x')\big)\, d\alpha,$$

la integral del gradiente a lo largo del segmento que va de x' a x [42, §3, ec. (1)].

Proposición 2.13 (Completitud de los gradientes integrados). *Si F es continua en el segmento $[x', x]$ y diferenciable en él salvo en un número finito de puntos, entonces*

$$\sum_{i=1}^{n} \mathrm{IG}_i(x; x') = F(x) - F(x').$$

En particular, los gradientes integrados cumplen la sensibilidad (a).

En el ejemplo 2.11, $\mathrm{IG}(2; 0) = 2\int_0^1 F'(2\alpha)\, d\alpha = 2\int_0^{1/2} 1\, d\alpha = 1 = F(2) - F(0)$. Los gradientes integrados dependen solo de las derivadas de la función, no de cómo se implementa, y por eso cumplen también la invariancia ante la implementación. Sundararajan *et al.* demuestran además que son el único método de «camino» que respeta la simetría entre variables [42, §4.2, teorema 1]. En la práctica, la integral se aproxima con una suma de Riemann de entre 20 y 300 pasos, y se comprueba que las atribuciones suman aproximadamente $F(x) - F(x')$ [42, §5].

2.6 Valores de Shapley y SHAP

Los gradientes integrados reparten un cambio continuo. Los valores de Shapley, de la teoría de juegos cooperativos, reparten el valor de un conjunto de «jugadores» que pueden estar presentes o ausentes.

Definición 2.14 (Valor de Shapley). Sea $N = \{1, \dots, M\}$ y sea $v : 2^N \to \mathbb{R}$ con $v(\varnothing) = 0$, un *juego cooperativo*. El *valor de Shapley* del jugador i es

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!\,(M - |S| - 1)!}{M!} \big(v(S \cup \{i\}) - v(S)\big).$$

Proposición 2.15 (Forma por permutaciones y eficiencia). *Sea Π el conjunto de las $M!$ ordenaciones de N y, para $\pi \in \Pi$, sea P_i^π el conjunto de jugadores que preceden a i en π. Entonces*

$$\phi_i(v) = \frac{1}{M!} \sum_{\pi \in \Pi} \big(v(P_i^\pi \cup \{i\}) - v(P_i^\pi)\big), \qquad \sum_{i=1}^{M} \phi_i(v) = v(N).$$

Los valores de Shapley cumplen además la *simetría* (dos jugadores intercambiables reciben lo mismo), el *jugador nulo* (quien no cambia ninguna coalición recibe 0) y la *linealidad* en v; el teorema de Shapley afirma que son la única asignación con estas cuatro propiedades.

Definición 2.17 (SHAP). Sea f un modelo, x una entrada con M características y, para $S \subseteq N$, sea $f_x(S)$ el valor esperado de f cuando se fijan las características de S a sus valores en x y las demás se toman de una distribución de referencia. Los *valores SHAP* de x son los valores de Shapley del juego $v(S) = f_x(S) - f_x(\varnothing)$ [32, §2 y teorema 1].

Lundberg y Lee demuestran que, entre las explicaciones aditivas $g(z') = \phi_0 + \sum_i \phi_i z'_i$, los valores SHAP son la única que cumple sus propiedades de precisión local, ausencia y consistencia [32, teorema 1]. En un modelo de lenguaje, los jugadores pueden ser los tokens del contexto y «ausente» puede significar sustituirlos por un token neutro. El coste exacto crece como 2^M, por lo que se usan aproximaciones por muestreo o el método *Kernel SHAP*.

Proposición 2.18 (SHAP de un modelo lineal). *Si $f(x) = w_0 + \sum_i w_i x_i$ y la distribución de referencia tiene características independientes con medias $\mathbb{E}[X_i]$, entonces $\phi_i = w_i\big(x_i - \mathbb{E}[X_i]\big)$.*

Tema 3

Superposición

Las neuronas de los modelos de lenguaje responden a menudo a varias cosas sin relación aparente. Este tema explica el fenómeno mediante la *hipótesis de superposición*: una red puede representar más características que dimensiones si las coloca en direcciones casi ortogonales y las características rara vez aparecen a la vez. Los resultados geométricos siguen el capítulo 2 de Blum, Hopcroft y Kannan [6], libro del curso CS 4850 de Cornell, y el lema del casquete esférico de Ball [3]. El modelo de juguete, la dimensión por característica y la transición de fase proceden de Elhage *et al.* [15], secciones «Definitions and Motivation», «Demonstrating Superposition» y «The Geometry of Superposition».

3.1 Características, polisemanticidad y bases privilegiadas

Definición 3.1 (Característica y polisemanticidad). Se llama *característica* a una propiedad de la entrada que la red representa mediante una dirección $W_i \in \mathbb{R}^m$ de su espacio de activaciones: la activación es aproximadamente $h = \sum_i x_i W_i$, donde $x_i \geq 0$ es la intensidad de la característica i. Una neurona (una coordenada de h) es *monosemántica* si responde a una sola característica y *polisemántica* si responde a varias [15, «What are Features?» y «Features as Directions»].

Que las coordenadas de un espacio tengan o no un significado especial depende de las operaciones que se aplican sobre él.

Proposición 3.2 (El flujo residual no tiene base privilegiada). *Sea un Transformer sin normalizaciones en el que el flujo residual solo se usa mediante productos por matrices (lecturas $W_{\text{in}}x$, escrituras $x \leftarrow x + W_{\text{out}}h$, embedding W_E y desembedding W_U). Sea $R \in \mathbb{R}^{d\times d}$ ortogonal. Si se sustituyen W_E por RW_E, cada W_{out} por RW_{out}, cada W_{in} por $W_{\text{in}}R^\top$ y W_U por $W_U R^\top$, la función que calcula el modelo no cambia y el nuevo flujo es Rx_ℓ en cada capa.*

Las coordenadas del flujo residual son, por tanto, arbitrarias: cualquier rotación da un modelo equivalente. En cambio, la capa oculta de un MLP aplica una no linealidad coordenada a coordenada, y $\mathrm{ReLU}(Rv) \neq R\,\mathrm{ReLU}(v)$ en general: con R la rotación de $90°$ en $\mathbb{R}^2$ y $v = (0, 1)$, $\mathrm{ReLU}(Rv) = \mathrm{ReLU}(-1, 0) = (0, 0)$, mientras que $R\,\mathrm{ReLU}(v) = Rv = (-1, 0)$. Esa capa tiene una *base privilegiada*, y en ella tiene sentido preguntar si una neurona es monosemántica [15, «Privileged vs Non-privileged Bases»]. En el flujo residual, la unidad de análisis natural es la dirección.

3.2 Casi ortogonalidad en alta dimensión

En $\mathbb{R}^m$ caben a lo sumo m direcciones exactamente ortogonales. Si se admite una pequeña interferencia, caben muchas más.

Teorema 3.3 (Coseno de un vector aleatorio). *Sea u un vector aleatorio con distribución uniforme en la esfera unidad $\mathbb{S}^{d-1} \subset \mathbb{R}^d$ y sea $v \in \mathbb{S}^{d-1}$ fijo. Entonces $\mathbb{E}[u^\top v] = 0$ y $\mathbb{E}[(u^\top v)^2] = 1/d$.*

La desviación típica del coseno es, por tanto, $1/\sqrt{d}$: con $d = 4096$ vale $1/64 = 0{,}016$. El resultado siguiente, que se enuncia sin demostración, cuantifica la concentración.

Lema 3.4 (Casquete esférico). *Con la notación del teorema 3.3, para $0 \le \varepsilon < 1$,*

$$P\big(|u^\top v| \ge \varepsilon\big) \le 2\, e^{-d\varepsilon^2/2}.$$

La cota de un solo lado, $P(u^\top v \ge \varepsilon) \le e^{-d\varepsilon^2/2}$, es la estimación del área de un casquete esférico de Ball [3]; la de dos lados se obtiene sumando la del lado opuesto, que es igual por simetría. Blum, Hopcroft y Kannan obtienen resultados del mismo tipo para la bola unidad [6, cap. 2].

Corolario 3.5 (Muchas direcciones casi ortogonales). *Sean $d \ge 1$, $0 < \varepsilon < 1$ y $N \ge 2$ con $N < e^{d\varepsilon^2/4}$. Existen N vectores unitarios de $\mathbb{R}^d$ con $|u_i^\top u_j| < \varepsilon$ para todo $i \ne j$.*

Con $d = 4096$ y $\varepsilon = 0{,}1$, el corolario garantiza $e^{10{,}24} \approx 2{,}8 \cdot 10^4$ direcciones; con $\varepsilon = 0{,}2$, $e^{40{,}96} \approx 6 \cdot 10^{17}$. El número de direcciones casi ortogonales crece exponencialmente con la dimensión. La cota es pesimista, pero basta para mostrar que la geometría no limita el número de características a d.

3.3 El modelo de juguete

Definición 3.6 (Modelo de juguete de superposición). Sean $n > m$. Cada característica x_i, $i = 1, \dots, n$, vale 0 con probabilidad S (la *dispersión*) y, en caso contrario, sigue una distribución uniforme en $[0, 1]$, independientemente de las demás. Sean $W \in \mathbb{R}^{m \times n}$, con columnas $W_1, \dots, W_n$, y $b \in \mathbb{R}^n$. El modelo comprime y reconstruye:

$$h = Wx \in \mathbb{R}^m, \qquad \hat{x} = \mathrm{ReLU}\big(W^\top h + b\big) = \mathrm{ReLU}\big(W^\top W x + b\big),$$

y se entrena con la pérdida $\mathcal{L} = \mathbb{E}\big[\sum_i I_i (x_i - \hat{x}_i)^2\big]$, donde $I_i > 0$ es la *importancia* de la característica i [15, «Experiment Setup»].

La columna W_i es la dirección asignada a la característica i, y la entrada $(W^\top W)_{ij} = W_i^\top W_j$ mide la *interferencia* entre las características i y j. Sin la ReLU, el modelo es lineal, $W^\top W$ tiene rango a lo sumo m y, según Elhage *et al.*, el modelo lineal aprende siempre las m características más importantes, de forma análoga al análisis de componentes principales; la superposición no le ofrece ninguna ventaja. La ReLU con sesgo negativo cambia esa conclusión: anula las interferencias pequeñas y negativas.

El ejemplo muestra el compromiso. Si las características son dispersas, casi siempre hay a lo sumo una activa, la interferencia se paga rara vez y guardar cinco características en dos dimensiones compensa. Si suelen activarse juntas, el error del tercer caso domina y conviene guardar solo dos. En la práctica del módulo, con $n = 5$, $m = 2$ e $I_i = 0{,}9^i$, el modelo entrenado guarda dos características ortogonales con $S = 0$, cuatro (dos pares antipodales perpendiculares) con $S = 0{,}7$ y las cinco en un pentágono con $S = 0{,}95$.

3.4 Dimensión por característica

Definición 3.8 (Dimensión por característica). Para $W_i \neq 0$ y $\hat{W}_i = W_i/\|W_i\|$, la *dimensión de la característica* i es

$$D_i = \frac{\|W_i\|^2}{\sum_{j=1}^{n} (\hat{W}_i^\top W_j)^2},$$

y se toma $D_i = 0$ si $W_i = 0$ [15, «Feature Dimensionality»]. La cantidad $m/\|W\|_F^2$ se llama *dimensiones por característica* del modelo.

El numerador mide cuánto se representa la característica y el denominador, cuántas características comparten su dirección. Una característica con dirección propia, ortogonal a las demás, tiene $D_i = 1$; dos características antipodales ($W_2 = -W_1$) tienen $D_i = 1/2$; en el pentágono, $D_i = 1/(1 + 2 \cdot 0{,}309^2 + 2 \cdot 0{,}809^2) = 1/2{,}5 = 0{,}4$. Elhage *et al.* observan que, en los modelos entrenados, los valores de D_i se agrupan en fracciones como $3/4$ (tetraedro), $2/3$ (triángulo), $1/2$ (par antipodal), $2/5$ (pentágono) y $3/8$ (antiprisma cuadrado).

Proposición 3.9 (Las dimensiones suman a lo sumo m). *Sea $W \in \mathbb{R}^{m \times n}$. Entonces $\sum_{i=1}^{n} D_i \leq \operatorname{rango} W \leq m$.*

3.5 Superposición como transición de fase

Elhage *et al.* estudian el caso mínimo, con dos características y una sola dimensión oculta, y lo representan en un diagrama de fases en función de la dispersión y de la importancia relativa [15, «Superposition as a Phase Change»]. Con importancias iguales hay dos candidatas naturales: guardar una sola característica y aproximar la otra por su media, o guardar las dos en direcciones opuestas ($w = (1, -1)$). La pérdida de cada solución es un polinomio sencillo en $q = 1 - S$, y las dos curvas se cortan en un valor crítico de la dispersión (ejercicio 3.7). Como el óptimo salta de una solución a otra cuando las curvas se cruzan, el cambio de geometría es abrupto, del tipo de una transición de fase de primer orden.

Observación 3.11. Si los modelos de lenguaje representan en superposición muchas más características que dimensiones, las neuronas serán polisemánticas y las proyecciones sobre direcciones arbitrarias mezclarán características. Elhage *et al.* señalan además una relación con la vulnerabilidad a perturbaciones adversarias: la interferencia permite que pequeñas perturbaciones en muchas direcciones se sumen [15, «Relationship to Adversarial Robustness»]. Encontrar una base sobrecompleta de características es el objetivo de los autoencoders dispersos del tema 4.

Tema 4

Autoencoders dispersos

Si las activaciones de un modelo son superposiciones de muchas características dispersas (tema 3), cabe intentar recuperarlas aprendiendo un diccionario sobrecompleto de direcciones tal que cada activación se escriba como combinación de unas pocas. Este tema formula ese problema, estudia el efecto de la penalización L^1 y describe los resultados publicados. El autoencoder disperso y sus métricas siguen a Bricken *et al.* [7] y a Templeton *et al.* [43], de Anthropic; la variante JumpReLU, a Lieberum *et al.* [30], de Google DeepMind. El umbral suave y la interpretación bayesiana de la penalización L^1 remiten al módulo 8, al §3.7 de Zhang *et al.* [46] y al curso CS229 de Stanford [10].

4.1 Aprendizaje de diccionarios

Definición 4.1 (Diccionario y código disperso). Sea $x \in \mathbb{R}^d$ una activación. Un *diccionario* es una familia de F direcciones $d_1, \dots, d_F \in \mathbb{R}^d$, con $F \geq d$ (diccionario *sobrecompleto* si $F > d$). Un *código* es un vector $f \in \mathbb{R}^F_{\geq 0}$ con $x \approx \sum_j f_j d_j$. Se definen $\|f\|_0 = |\{j : f_j \neq 0\}|$, el número de latentes activos, y $\|f\|_1 = \sum_j f_j$.

Con $F > d$, la ecuación $x = \sum_j f_j d_j$ tiene en general infinitas soluciones. Se busca la más dispersa, con pocos f_j no nulos. Minimizar $\|f\|_0$ es un problema combinatorio; la penalización $\|f\|_1$ es su relajación convexa habitual, ya usada en la regresión LASSO del módulo 8.

4.2 El autoencoder disperso

Definición 4.2 (Autoencoder disperso). Sean $W_e \in \mathbb{R}^{d \times F}$, $b_e \in \mathbb{R}^F$, $W_d \in \mathbb{R}^{F \times d}$ con filas $d_1^\top, \dots, d_F^\top$, y $b_d \in \mathbb{R}^d$. El *autoencoder disperso* (SAE) calcula

$$f(x) = \mathrm{ReLU}\big(W_e^\top (x - b_d) + b_e\big) \in \mathbb{R}^F, \qquad \hat{x} = W_d^\top f(x) + b_d = \sum_{j=1}^{F} f_j(x)\, d_j + b_d,$$

y se entrena sobre activaciones del modelo con la pérdida

$$\mathcal{L}(x) = \|x - \hat{x}\|_2^2 + \lambda \sum_{j=1}^{F} f_j(x)\, \|d_j\|_2,$$

donde $\lambda > 0$ es el *coeficiente de dispersión*. La fila d_j es la dirección que escribe el latente j [7, 43].

Proposición 4.3 (El factor $\|d_j\|$ evita una trampa de escala). *Sea $c > 0$ y considérese la transformación $W_e \mapsto W_e/c$, $b_e \mapsto b_e/c$, $d_j \mapsto c\, d_j$ para todo j, que deja b_d fijo.*

(i) La transformación no cambia la reconstrucción $\hat{x}$ y multiplica cada f_j por $1/c$.

(ii) Si la penalización fuera $\lambda \sum_j f_j$, el valor de la penalización se dividiría por c; tomando $c \to \infty$, el ínfimo de la pérdida sería el de la reconstrucción sola y la penalización no tendría efecto.

(iii) Con la penalización $\lambda \sum_j f_j \|d_j\|$ de la definición 4.2, la pérdida es invariante frente a la transformación.

Una alternativa equivalente, usada por Bricken *et al.*, es imponer $\|d_j\| = 1$ durante el entrenamiento.

4.3 La penalización L^1: umbral suave y encogimiento

Proposición 4.5 (Umbral suave). *Sean $a \in \mathbb{R}$ y $\lambda > 0$. El problema $\min_{f \geq 0} g(f)$, con $g(f) = (a - f)^2 + \lambda f$, tiene solución única*

$$f^* = \max\bigl(a - \tfrac{\lambda}{2},\ 0\bigr).$$

La penalización L^1 tiene dos efectos: anula las activaciones menores que $\lambda/2$, lo que produce la dispersión, y *encoge* en $\lambda/2$ las que sobreviven. El encogimiento es un sesgo sistemático: el SAE subestima la intensidad de las características activas. La forma $\max(a - \lambda/2, 0) = \mathrm{ReLU}(a - \lambda/2)$ coincide con la de un codificador con sesgo negativo, lo que explica la elección del codificador de la definición 4.2 (ejercicio 4.6). Para reducir el encogimiento se han propuesto otras activaciones; Gemma Scope usa *JumpReLU*, que pone a cero las preactivaciones por debajo de un umbral aprendido y deja pasar las demás sin restarles nada [30].

La penalización tiene también una interpretación bayesiana: minimizar el error cuadrático más $\lambda\|f\|_1$ equivale a buscar el código más probable *a posteriori* con ruido gaussiano en la reconstrucción y una distribución *a priori* de Laplace (exponencial, si $f \geq 0$) para cada latente (ejercicio 4.7). Suponer que las características son dispersas es elegir esa distribución *a priori*.

4.4 Métricas de un diccionario

Definición 4.6 (Métricas de un SAE). Sean $x^{(1)}, \ldots, x^{(N)}$ activaciones con media $\bar{x}$, y sean $\hat{x}^{(k)}$ y $f^{(k)}$ sus reconstrucciones y códigos.

(i) La *fracción de varianza no explicada* es $\mathrm{FVU} = \sum_k \|x^{(k)} - \hat{x}^{(k)}\|^2 / \sum_k \|x^{(k)} - \bar{x}\|^2$.

(ii) El *L0 medio* es $\frac{1}{N}\sum_k \|f^{(k)}\|_0$, el número medio de latentes activos.

(iii) Un latente j está *muerto* si $f_j^{(k)} = 0$ para todo k en un conjunto amplio de datos.

(iv) La *pérdida recuperada* compara la entropía cruzada del modelo cuando se sustituye x por $\hat{x}$ con la del modelo original y con la del modelo con la activación anulada.

Ninguna métrica basta sola. Un FVU bajo con un L0 alto indica que el SAE reconstruye bien pero no descompone; un L0 bajo con un FVU alto indica que descompone pero pierde información. La pérdida recuperada mide lo que importa para el modelo: un error de reconstrucción pequeño en una dirección que el modelo lee puede ser más dañino que uno grande en una dirección que ignora.

4.5 Resultados publicados

- *Towards Monosemanticity* (Bricken *et al.*, 2023) [7] entrena SAE sobre las 512 neuronas del MLP de un Transformer de una capa, con diccionarios de hasta 131 072 latentes. Los latentes son mucho más interpretables que las neuronas: aparecen características para texto en árabe, secuencias de ADN o codificación base64. El trabajo documenta los latentes muertos y el fenómeno de *división de características*: al aumentar F, una característica se parte en varias más específicas.
- *Scaling Monosemanticity* (Templeton *et al.*, 2024) [43] entrena SAE sobre el flujo residual de una capa intermedia de Claude 3 Sonnet, con 1, 4 y 34 millones de latentes. Encuentra características abstractas, multilingües y multimodales (el puente Golden Gate en texto y en imágenes) y otras relevantes para la seguridad, como el engaño, la adulación o el código inseguro. Fijar una característica en un valor alto cambia la conducta: con la del puente activada, el modelo lo menciona en casi cualquier respuesta, demostración pública conocida como *Golden Gate Claude* (mayo de 2024).
- *Gemma Scope* (Lieberum *et al.*, 2024) [30] publica SAE JumpReLU abiertos para todas las capas y subcapas de Gemma 2 2B y 9B y para algunas capas de Gemma 2 27B.

Observación 4.8 (Límites). Siempre queda varianza sin explicar; el diccionario obtenido depende de λ y de F, y un latente interpretable no es necesariamente una variable que el modelo use. Como en las sondas (observación 2.5), la prueba de que una característica es causal exige intervenir sobre ella (tema 5).

Tema 5

Circuitos e intervenciones causales

Las sondas y los diccionarios dicen qué información hay en las activaciones; este tema estudia cómo se procesa y cómo se demuestra que un mecanismo es causal. Un *circuito* es un subgrafo de componentes y pesos que implementa un algoritmo identificable [37]. Se estudian cuatro casos publicados: las cabezas de inducción de Elhage *et al.* [14, «Two-Layer Attention-Only Transformers»] y Olsson *et al.* [38], también tratadas en el §10.6 de Jurafsky y Martin [25]; el circuito de identificación del objeto indirecto de Wang *et al.* [45]; la localización y edición de hechos de Meng *et al.* [33], y el algoritmo de Fourier del *grokking* de Nanda *et al.* [35]. El tema termina con los vectores de dirección [44, 2] y el trazado de circuitos en Claude 3.5 Haiku [31]. La deducción de la actualización de ROME usa el problema de mínimos cuadrados con restricciones de la clase 16 del curso ENGR108 de Stanford [16].

5.1 Composición de cabezas y cabezas de inducción

En un Transformer de dos capas, la cabeza de la segunda capa lee un flujo que ya contiene la salida de la primera. Con $x_j = x_{0,j} + h^1_j$, sus puntuaciones son

$$\sqrt{d_h}\, s^2_{ij} = x_{0,i}^\top W^2_{QK} x_{0,j} + \underbrace{x_{0,i}^\top W^2_{QK} h^1_j}_{\text{composición K}} + \underbrace{(h^1_i)^\top W^2_{QK} x_{0,j}}_{\text{composición Q}} + (h^1_i)^\top W^2_{QK} h^1_j,$$

y su salida contiene el término $W^2_{OV} h^1_j$ (*composición V*; ejercicio 1.5). Elhage *et al.* llaman a estas tres vías las tres clases de composición [14, «Three Kinds of Composition»]. La composición Q y K permite patrones de atención que dependen de algo más que de los tokens de origen y destino.

Definición 5.1 (Cabeza de inducción). Una *cabeza de inducción* es una cabeza de atención que, sobre secuencias de la forma $\dots[A][B]\dots[A]$, atiende desde la última $[A]$ a la posición de $[B]$ (*coincidencia de prefijo*) y aumenta el logit de $[B]$ (*copia*) [38].

Proposición 5.2 (Un circuito de inducción explícito). *Sean $u_t = W_E e_t$ los embeddings de los tokens, ortonormales, y sean u'_t otros vectores ortonormales, ortogonales a todos los u_t. Sea $\Pi = \sum_t u'_t u_t^\top$, de modo que $\Pi u_t = u'_t$. Supóngase:*

(i) la cabeza 1 atiende siempre a la posición anterior ($A^1_{j,j-1} = 1$) y tiene $W^1_{OV} = \Pi$, de modo que $h^1_j = u'_{t_{j-1}}$ y el flujo tras la capa 1 es $x_j = u_{t_j} + u'_{t_{j-1}}$;

(ii) la cabeza 2 tiene $W^2_{QK} = \beta\sqrt{d_h}\,\Pi^\top$, con $\beta > 0$, y su circuito OV copia: $W_U W^2_{OV} u_t = \gamma\, e_t$ con $\gamma > 0$ y $W^2_{OV} u'_t = 0$.

Entonces la puntuación de la cabeza 2 desde la posición i hacia la posición j es $s^2_{ij} = \beta\,[t_{j-1} = t_i]$, y su aportación a los logits en la posición i es $\gamma \sum_j A^2_{ij} e_{t_j}$. Cuando $\beta \to \infty$, la atención se reparte por igual entre las posiciones j con $t_{j-1} = t_i$; en particular, si t_i ha aparecido antes una sola vez, seguido del token t_j, el logit de t_j aumenta en γ.

La proposición muestra por qué hace falta una cabeza previa: la clave de una posición contiene, de entrada, solo su propio token, y alguien debe escribir en ella cuál era el token anterior. La cabeza 2 lee esa información a través de la composición K.

Las cabezas de inducción reducen drásticamente la pérdida en texto repetido. Sobre una secuencia aleatoria de tokens de un vocabulario de $K = 50\,000$ que se repite, sin inducción la mejor predicción es la uniforme, con pérdida $\ln 50\,000 = 10{,}82$ nats por token; si en la repetición se copia el token correcto con probabilidad 0,9, la pérdida en esa parte baja a $-\ln 0{,}9 = 0{,}105$ nats.

Olsson *et al.* definen la *puntuación de aprendizaje en contexto* como la pérdida en el token 500 del contexto menos la pérdida media en el token 50, y observan que, en un intervalo corto del entrenamiento, esta puntuación cae bruscamente al mismo tiempo que se forman las cabezas de inducción y aparece un pequeño saliente en la curva de pérdida [38]. Los modelos de una sola capa no presentan ni las cabezas ni el salto. El trabajo reúne seis argumentos para sostener que las cabezas de inducción son responsables de buena parte del aprendizaje en contexto (módulo 22); los propios autores advierten que, para los modelos grandes, la evidencia es indirecta.

5.2 Intervenciones: ablación y parcheo de activaciones

Definición 5.3 (Ablación y parcheo). Sea M un modelo, c un componente (una cabeza, un MLP o una activación concreta) y μ una métrica de la salida, por ejemplo la diferencia de logits entre la respuesta correcta y una alternativa.

(i) La *ablación* de c sustituye su salida por cero (*ablación nula*) o por su media sobre un conjunto de datos (*ablación por la media*).

(ii) Dadas una entrada *limpia* u y una entrada *corrupta* u' que solo difiere en la información relevante, el *parcheo de activaciones* ejecuta el modelo sobre u' sustituyendo la salida de c por la que tenía sobre u, y mide μ.

(iii) La *fracción recuperada* es $\big(\mu_{\text{parcheo}} - \mu_{\text{corrupta}}\big)/\big(\mu_{\text{limpia}} - \mu_{\text{corrupta}}\big)$.

Meng *et al.* llaman *efecto total* a $\mu_{\text{limpia}} - \mu_{\text{corrupta}}$ y *efecto indirecto* de un estado oculto a la mejora que se obtiene restaurándolo [33, §2.1]. El parcheo distingue lo necesario de lo suficiente: si restaurar c recupera la conducta, c transporta la información decisiva. El *parcheo de caminos* de Wang *et al.* restaura solo el efecto de c a través de ciertos caminos (por ejemplo, el camino directo a los logits) y deja los demás corruptos [45, §3.1].

5.3 Localización y edición de hechos: ROME

Meng *et al.* aplican el parcheo a enunciados de hechos («La Torre Eiffel está en…») corrompiendo con ruido gaussiano los embeddings del sujeto. En GPT-2 XL, el efecto indirecto medio es máximo en los MLP de las capas intermedias, en el último token del sujeto [33, §2.2]. Proponen entonces que esos MLP funcionan como una *memoria asociativa* lineal: la matriz de salida W del MLP asocia una *clave* k (la activación oculta, que codifica el sujeto) con un *valor* $v = Wk$ (que codifica la relación y el objeto).

Teorema 5.5 (Actualización de rango uno). *Sean $K \in \mathbb{R}^{p\times N}$ y $V \in \mathbb{R}^{q\times N}$ matrices de claves y valores, con $C = KK^{\top}$ definida positiva, y sea W la solución de mínimos cuadrados*

$\min_W \|WK - V\|_F^2$. *Dada una nueva asociación* (k_*, v_*)*, la solución de*

$$\min_{\hat{W}} \|\hat{W}K - V\|_F^2 \quad \textit{sujeto a} \quad \hat{W}k_* = v_*$$

es

$$\hat{W} = W + \Lambda\,(C^{-1}k_*)^\top, \qquad \Lambda = \frac{v_* - Wk_*}{(C^{-1}k_*)^\top k_*}.$$

Es la ecuación (2) de Meng *et al.* [33, §3.1], que estiman C con una muestra de texto de Wikipedia. El cambio tiene rango uno y es mínimo en la métrica que define C: no altera las claves k con $k^\top C^{-1} k_* = 0$ (ejercicio 5.5). Meng *et al.* evalúan las ediciones con su conjunto CounterFact, que mide a la vez si la edición generaliza a paráfrasis y si es específica del sujeto [33, §3.3].

5.4 *Grokking* y el algoritmo de Fourier

Nanda *et al.* entrenan un Transformer de una capa para la suma modular: dados $a, b \in \{0, \dots, p-1\}$ con $p = 113$, predecir $c = (a + b) \bmod p$. Con el 30 % de los pares como entrenamiento y decaimiento de pesos, la red memoriza primero y generaliza mucho después (*grokking*). La ingeniería inversa muestra que implementa el algoritmo siguiente [35, §3.1 y §4]: los embeddings representan cada entrada mediante $\cos(\omega_k a)$ y $\sin(\omega_k a)$ para unas pocas frecuencias $\omega_k = 2\pi k/p$; la atención y el MLP calculan $\cos(\omega_k(a+b))$ y $\sin(\omega_k(a+b))$ con las identidades trigonométricas

$$\cos(\omega(a+b)) = \cos\omega a \cos\omega b - \sin\omega a \sin\omega b, \qquad \sin(\omega(a+b)) = \sin\omega a \cos\omega b + \cos\omega a \sin\omega b;$$

y la matriz de salida produce el logit

$$\ell(c) = \sum_{k} \cos(\omega_k(a + b - c)) = \sum_{k} \big[\cos(\omega_k(a+b))\cos(\omega_k c) + \sin(\omega_k(a+b))\sin(\omega_k c)\big],$$

que es lineal en las cantidades anteriores. El modelo estudiado usa cinco frecuencias clave, $k \in \{14, 35, 41, 42, 52\}$.

Proposición 5.7 (Interferencia constructiva). *Sea p primo y sea $\mathcal{K} \subseteq \{1, \dots, p-1\}$ no vacío. La función $\ell(c) = \sum_{k \in \mathcal{K}} \cos\big(2\pi k(a + b - c)/p\big)$, para $c \in \{0, \dots, p-1\}$, alcanza su máximo, igual a $|\mathcal{K}|$, únicamente en $c \equiv a + b \pmod p$.*

Nanda *et al.* definen además medidas de progreso que evolucionan de forma continua antes del salto en la exactitud de prueba, y distinguen tres fases: memorización, formación del circuito y limpieza, en la que el decaimiento de pesos elimina los componentes que memorizan [35, §5]. El *grokking* no es, por tanto, un cambio súbito del mecanismo, sino la culminación de un proceso gradual.

5.5 Vectores de dirección

Definición 5.8 (Dirección por adición de activaciones). Sean P y N conjuntos de entradas con y sin un concepto, y $x_\ell(u)$ el flujo de la capa ℓ. El *vector de dirección* es la diferencia de medias

$$v = \frac{1}{|P|}\sum_{u \in P} x_\ell(u) - \frac{1}{|N|}\sum_{u \in N} x_\ell(u),$$

y *dirigir* el modelo consiste en sustituir $x_\ell \leftarrow x_\ell + \alpha v$ durante la generación [44].

Si la dirección se suma justo antes de W_U y no hay normalización, los logits cambian en $\alpha W_U v$, de modo que la log-probabilidad de cada token se desplaza en $\alpha(W_U v)_t$ menos una constante común. En capas anteriores el efecto pasa por las capas posteriores y solo puede medirse experimentalmente. Con α excesivo, el flujo sale de la distribución que el modelo ha visto en el entrenamiento y la generación degenera.

La operación inversa, la *ablación de una dirección*, sustituye $x \leftarrow x - (\hat{v}^\top x)\hat{v}$ con $\hat{v} = v/\|v\|$, y elimina del flujo la componente según $\hat{v}$. Arditi *et al.* muestran que en varios modelos abiertos con ajuste de seguridad la negativa a responder peticiones dañinas está mediada por una sola dirección: al eliminarla, el modelo deja de negarse, y al sumarla, se niega incluso ante peticiones inocuas [2]. El resultado ilustra a la vez la potencia de las intervenciones y la fragilidad de algunas salvaguardas. Li *et al.* usan intervenciones de este tipo en cabezas de atención para aumentar la veracidad de las respuestas [29], y *Golden Gate Claude* (tema 4) es una intervención sobre una característica de un SAE.

5.6 Trazado de circuitos en un modelo comercial

En 2025, Anthropic publicó el análisis de Claude 3.5 Haiku mediante *grafos de atribución* [31]. Se sustituyen los MLP por un *transcodificador* entre capas, con unos 30 millones de características interpretables, que forma un modelo de reemplazo; se calcula qué características influyen en cuáles para una entrada concreta, y las hipótesis se validan con intervenciones sobre el modelo original. Entre los casos documentados figuran el razonamiento en varios pasos (de «Dallas» a «Texas» y de ahí a «Austin»), la elección de la palabra que rimará antes de escribir un verso, circuitos compartidos entre idiomas y un circuito de «no puedo responder» que se inhibe al reconocer una entidad conocida y cuyo fallo produce alucinaciones. Es información pública de Anthropic; sobre los mecanismos internos de Gemini o de Grok no constan, en la información pública consultada, análisis equivalentes.

Tema 6

Evaluación, calibración y seguridad

Este tema estudia la conducta de un modelo desde fuera: cómo medir su rendimiento con conjuntos de pruebas, cómo cuantificar la incertidumbre de esas medidas, cuándo su confianza es fiable, por qué alucina y cómo se buscan y se miden sus fallos de seguridad. La organización sigue la clase 11, «Benchmarking and Evaluation», de CS224N [8] y la semana de robustez adversaria de CS 2881r de Harvard [11]. Los conjuntos de evaluación proceden de Hendrycks *et al.* [21], Liang *et al.* [20] y Chen *et al.* [13]; la calibración, de Guo *et al.* [19] y Kadavath *et al.* [26]; las alucinaciones, de Kalai *et al.* [28], y los ataques, de Zou *et al.* [47] y Ganguli *et al.* [17]. Los intervalos de confianza siguen la clase 23 del curso 18.05 del MIT [34].

6.1 Conjuntos de evaluación y su incertidumbre

Un conjunto de evaluación (*benchmark*) es una colección fija de tareas con un criterio automático de acierto.

- *MMLU* [21] reúne preguntas de opción múltiple de 57 materias, desde matemáticas elementales hasta derecho; en su publicación, el mayor GPT-3 superaba el azar en casi 20 puntos porcentuales de media.
- *HELM* [20] evalúa 30 modelos en 16 escenarios básicos con siete métricas (exactitud, calibración, robustez, equidad, sesgo, toxicidad y eficiencia), además de 26 escenarios específicos, y eleva la fracción de combinaciones de modelo y escenario evaluadas del 17,9 % al 96,0 %.
- *HumanEval* [13] contiene 164 problemas de programación escritos a mano, cada uno con su firma, su descripción y pruebas unitarias; una solución es correcta si supera todas las pruebas.

Definición 6.1 (Exactitud empírica). Sea un modelo con probabilidad de acierto θ en una población de tareas, y sean n tareas extraídas de forma independiente. Sea $X_k \in \{0, 1\}$ el acierto en la tarea k. La *exactitud empírica* es $\bar{X} = \frac{1}{n}\sum_k X_k$.

Proposición 6.2 (Intervalo conservador). *Con la notación de la definición 6.1,* $\mathbb{E}[\bar{X}] = \theta$ *y* $\mathrm{Var}(\bar{X}) = \theta(1-\theta)/n \le 1/(4n)$. *En consecuencia, por la aproximación normal,* $\bar{X} \pm z_{\alpha/2}/(2\sqrt{n})$ *es un intervalo de confianza aproximado de nivel al menos* $1-\alpha$ *para* θ*; con* $z_{0{,}025} \approx 2$*, el intervalo al* 95 % *es* $\bar{X} \pm 1/\sqrt{n}$.

Con $n = 1000$ preguntas, el margen al 95 % es $1{,}96/(2\sqrt{1000}) = 0{,}031$: dos modelos con exactitudes del 70 % y del 72 % no se distinguen con esa muestra (ejercicio 6.5). Las diferencias de uno o dos puntos que se publican a menudo exigen conjuntos de varios miles de preguntas, o comparaciones emparejadas sobre las mismas preguntas. A la incertidumbre estadística se suma la *contaminación* (módulo 22): si las preguntas aparecieron en los datos de entrenamiento, la exactitud mide memoria y no capacidad.

6.2 pass@k y su estimador insesgado

Definición 6.3 (pass@k). Sea un problema que el modelo resuelve en cada intento, de forma independiente, con probabilidad p. Su *pass@k* es la probabilidad de que al menos una de k muestras sea correcta, $1-(1-p)^k$. El pass@k de un conjunto de problemas es la media de los pass@k de sus problemas [13, §2.1].

Calcular $1-(1-\hat{p})^k$ con una estimación $\hat{p}$ de p da un valor sesgado (ejercicio 6.6). Chen *et al.* generan $n \geq k$ muestras por problema, cuentan las c correctas y usan otro estimador.

Proposición 6.4 (Estimador insesgado de pass@k). *Sean $n \geq k$ muestras independientes, cada una correcta con probabilidad p, y sea c el número de correctas. Entonces*

$$\widehat{\text{pass}}_k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

cumple $\mathbb{E}\big[\widehat{\text{pass}}_k\big] = 1-(1-p)^k$, con el convenio $\binom{n-c}{k}=0$ si $n-c<k$. Además, si $n-c \geq k$,

$$\frac{\binom{n-c}{k}}{\binom{n}{k}} = \prod_{i=n-c+1}^{n}\Big(1-\frac{k}{i}\Big).$$

La forma de producto evita calcular números combinatorios enormes y es la implementación que proponen Chen *et al.* [13, §2.1, fig. 3], que usan $n=200$ y $k \leq 100$.

6.3 Calibración

Definición 6.6 (Calibración, ECE y MCE). Sea un clasificador que devuelve una predicción $\hat{Y}$ y una confianza $\hat{P} \in [0,1]$. Está *perfectamente calibrado* si $P(\hat{Y}=Y \mid \hat{P}=q)=q$ para todo $q \in [0,1]$. Dadas n predicciones agrupadas en M intervalos de confianza $B_1,\dots,B_M$, sean acc_b la exactitud y conf_b la confianza media en B_b. El *error de calibración esperado* y el *error de calibración máximo* son

$$\text{ECE} = \sum_{b=1}^{M} \frac{|B_b|}{n}\,|\text{acc}_b - \text{conf}_b|, \qquad \text{MCE} = \max_b |\text{acc}_b - \text{conf}_b|$$

[19, §2, ecs. (1), (3) y (5)].

El diagrama de fiabilidad representa acc_b frente a conf_b; un modelo perfectamente calibrado sigue la diagonal. Guo *et al.* observan que las redes profundas modernas están peor calibradas que las antiguas y proponen, entre otros métodos, el *escalado de temperatura*.

Definición 6.8 (Escalado de temperatura). Dados los logits z de un clasificador, el escalado de temperatura devuelve $\text{softmax}(z/T)$, con un único parámetro $T>0$ ajustado minimizando la log-verosimilitud negativa en un conjunto de validación [19, §4].

Proposición 6.9 (El escalado de temperatura no cambia la exactitud). *Para todo $T>0$, $\arg\max_k \text{softmax}(z/T)_k = \arg\max_k z_k$.*

El escalado solo cambia la confianza, no la predicción: $T>1$ suaviza las probabilidades y corrige la sobreconfianza. Kadavath *et al.* muestran que los modelos de lenguaje grandes preentrenados están razonablemente bien calibrados en preguntas de opción múltiple con

el formato adecuado y que pueden estimar la probabilidad de conocer la respuesta [26]; el informe técnico de GPT-4 muestra que el posentrenamiento empeoró la calibración de su modelo preentrenado [18].

6.4 Alucinaciones y reglas de puntuación

Una *alucinación* es una afirmación falsa formulada con fluidez y seguridad. Kalai *et al.* [28] sostienen que no es un fallo misterioso, sino una consecuencia estadística del entrenamiento y de la evaluación.

- *Preentrenamiento.* Reducen la generación de respuestas válidas a un problema de clasificación binaria («¿es válida esta respuesta?») y prueban que la tasa de error generativo es al menos, aproximadamente, el doble de la tasa de error de esa clasificación [28, §3.1, corolario 1]. Para hechos arbitrarios, como la fecha de cumpleaños de una persona poco conocida, la tasa de alucinación de un modelo base está acotada inferiormente por la fracción de hechos que aparecen una sola vez en los datos de entrenamiento [28, §3.3.1, teorema 2], resultado anticipado por Kalai y Vempala [27].
- *Evaluación.* La mayoría de los conjuntos de evaluación puntúan 1 el acierto y 0 tanto el error como la abstención, de modo que adivinar nunca resta. Un modelo optimizado para esas puntuaciones aprende a no decir nunca «no lo sé».

Proposición 6.10 (Umbral de confianza explícito). *Sea $t \in [0, 1)$ y una regla de puntuación que da 1 punto por acierto, 0 por abstención y $-t/(1-t)$ por error. Si el modelo acierta con probabilidad p, responder tiene mayor puntuación esperada que abstenerse si y solo si $p > t$.*

Kalai *et al.* proponen incluir el umbral en las instrucciones de la evaluación, con valores como $t = 0{,}5$, 0,75 o 0,9; la puntuación binaria habitual corresponde a $t = 0$ [28, §4.2].

6.5 *Red teaming* y ataques adversarios

Una *evaluación de seguridad* mide, antes del despliegue, si un modelo tiene una capacidad peligrosa o si produce conductas dañinas cuando se le pide. El *red teaming* busca activamente entradas que provoquen fallos. Ganguli *et al.* reunieron y analizaron unos 39 000 ataques escritos por personas contra modelos de Anthropic [17]; Perez *et al.* usan un modelo de lenguaje para generar los ataques de forma automática [40].

Definición 6.11 (Ataque por sufijo adversario). Sea u una petición dañina, $y^{\ast} = (y_1^{\ast}, \dots, y_T^{\ast})$ el comienzo de una respuesta afirmativa (por ejemplo, «Claro, aquí tienes…») y s un sufijo de tokens. El ataque busca

$$s^{\ast} = \arg\min_{s} \; \mathcal{L}(s), \qquad \mathcal{L}(s) = -\sum_{t=1}^{T} \ln p\big(y_t^{\ast} \mid u \oplus s \oplus y_{<t}^{\ast}\big),$$

donde $\oplus$ denota concatenación [47, §2.1].

El problema es discreto. El algoritmo *Greedy Coordinate Gradient* (GCG) de Zou *et al.* calcula el gradiente de $\mathcal{L}$ respecto de la codificación *one-hot* de cada token del sufijo, toma para cada posición los k tokens con gradiente más negativo como candidatos, evalúa exactamente

la pérdida de un lote de sustituciones aleatorias entre esos candidatos y conserva la mejor [47, §2.2]. Optimizado sobre varias peticiones y varios modelos abiertos a la vez, el sufijo resultante se transfiere a modelos cerrados: en su evaluación, la tasa de éxito fue del 86,6 % en GPT-3.5, del 46,9 % en GPT-4, del 47,9 % en Claude 1, del 2,1 % en Claude 2 y del 66,0 % en PaLM-2 [47, §3.2]. Son cifras de 2023 sobre versiones ya retiradas; los proveedores corrigen estos ataques una vez publicados.

6.6 Estadística de los sucesos raros

Las conductas peligrosas son, por diseño, raras, y su medida requiere un cuidado estadístico particular.

Proposición 6.12 (Intentos repetidos). *Si un ataque tiene éxito en cada intento, de forma independiente, con probabilidad p, la probabilidad de al menos un éxito en k intentos es $1-(1-p)^k$, que es mayor o igual que $1-e^{-kp}$.*

Con $p = 0{,}02$, el ataque triunfa al menos una vez en 100 intentos con probabilidad $1 - 0{,}98^{100} = 0{,}867$. Un ataque que parece raro en una prueba es casi seguro para un atacante persistente.

Proposición 6.13 (Regla del 3). *Sean n pruebas independientes sin ningún fallo, cada una con probabilidad de fallo p desconocida. La cota superior exacta de confianza $1-\alpha$ para p, es decir, el valor p_u con $(1-p_u)^n = \alpha$, cumple $p_u \le \ln(1/\alpha)/n$. Para $\alpha = 0{,}05$, $\ln 20 = 2{,}996$ y $p_u \le 3/n$.*

Con 300 pruebas sin fallos, la tasa de fallo es menor que $3/300 = 1\,\%$ con confianza del 95 %; la cota exacta es $1 - 0{,}05^{1/300} = 0{,}99\,\%$. Cero fallos no significa tasa cero.

6.7 Marcos de seguridad y conductas ocultas

Las evaluaciones de seguridad suelen agruparse en cuatro preguntas: si el modelo *puede* hacer algo peligroso (capacidades, como la ayuda en ciberataques o en armas biológicas), si *lo hará* cuando se le pida (salvaguardas, como la tasa de éxito de los ataques), si *persigue* lo que se le pidió (alineamiento: adulación, engaño, sabotaje de tareas) y si *dice lo que representa internamente* (honestidad: calibración y alucinaciones). La *Responsible Scaling Policy* de Anthropic [41] define niveles de seguridad (ASL) que exigen medidas de protección más estrictas cuando las evaluaciones detectan ciertas capacidades; Google DeepMind y OpenAI han publicado marcos análogos.

Hay dos límites conocidos de las evaluaciones de conducta. El primero es que pueden no reflejar el despliegue: contaminación, jueces automáticos sesgados y modelos que distinguen cuándo se les evalúa. El segundo es más profundo: Hubinger *et al.* entrenaron deliberadamente modelos con puertas traseras (por ejemplo, escribir código seguro si el año indicado es 2023 e insertar vulnerabilidades si es 2024) y comprobaron que la conducta persistía tras el ajuste supervisado, el aprendizaje por refuerzo y el entrenamiento adversario, sobre todo en los modelos más grandes; el entrenamiento adversario enseñaba a reconocer mejor el desencadenante y a ocultar la conducta [24]. Una evaluación de conducta solo ve entradas y salidas; por eso se combina con las herramientas de interpretabilidad de los temas 1 a 5, que miran dentro.

Bibliografía

[1] G. Alain e Y. Bengio. Understanding intermediate layers using linear classifier probes. arXiv:1610.01644, 2016. https://arxiv.org/abs/1610.01644

[2] A. Arditi, O. Obeso, A. Syed, D. Paleka, N. Panickssery, W. Gurnee y N. Nanda. Refusal in language models is mediated by a single direction. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. https://arxiv.org/abs/2406.11717

[3] K. Ball. An elementary introduction to modern convex geometry. En S. Levy (ed.), *Flavors of Geometry*, MSRI Publications 31. Cambridge University Press, 1997.

[4] N. Belrose, Z. Furman, L. Smith, D. Halawi, I. Ostrovsky, L. McKinney, S. Biderman y J. Steinhardt. Eliciting latent predictions from transformers with the tuned lens. arXiv:2303.08112, 2023. https://arxiv.org/abs/2303.08112

[5] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. §4.2 «Probabilistic Generative Models».

[6] A. Blum, J. Hopcroft y R. Kannan. *Foundations of Data Science*. Cambridge University Press, 2020. Cap. 2, «High-Dimensional Space». Texto del curso CS 4850 de Cornell. https://www.cs.cornell.edu/jeh/book.pdf

[7] T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly *et al.* Towards monosemanticity: decomposing language models with dictionary learning. *Transformer Circuits Thread*, Anthropic, 2023. https://transformer-circuits.pub/2023/monosemantic-features/index.html

[8] D. Yang y Y. Choi. CS224N: Natural Language Processing with Deep Learning. Stanford University, invierno de 2026. Clase 11, «Benchmarking and Evaluation»; clase invitada de B. Kim, «Interpretability». https://web.stanford.edu/class/cs224n/

[9] Stanford CS229: Machine Learning. *Problem Set* 1. Stanford University, verano de 2019. https://cs229.stanford.edu/summer2019/ps1.pdf

[10] Stanford CS229: Machine Learning. *Problem Set* 2. Stanford University, verano de 2020. Problema 6, «Bayesian Interpretation of Regularization». https://cs229.stanford.edu/summer2020/ps2.pdf

[11] B. Barak. CS 2881r: AI Safety. Harvard University, otoño de 2025. Semana 3, «Adversarial Robustness, Jailbreaks, Prompt Injection, Security». https://windowsontheory.org/2025/09/10/cs-2881-ai-safety/

[12] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2026. Práctica 1, «Basics», versión 26.0.3. https://github.com/stanford-cs336/assignment1-basics

[13] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan *et al.* Evaluating large language models trained on code. arXiv:2107.03374, 2021. https://arxiv.org/abs/2107.03374

[14] N. Elhage, N. Nanda, C. Olsson, T. Henighan, N. Joseph, B. Mann *et al.* A mathematical framework for transformer circuits. *Transformer Circuits Thread*, Anthropic, 2021. https://transformer-circuits.pub/2021/framework/index.html

[15] N. Elhage, T. Hume, C. Olsson, N. Schiefer, T. Henighan *et al.* Toy models of superposition. *Transformer Circuits Thread*, Anthropic, 2022. También arXiv:2209.10652. https://transformer-circuits.pub/2022/toy_model/index.html

[16] S. Boyd. ENGR108: Introduction to Matrix Methods. Stanford University. Clase 16, «Constrained least squares». Texto: S. Boyd y L. Vandenberghe, *Introduction to Applied Linear Algebra*, Cambridge University Press, 2018. https://web.stanford.edu/class/engr108/lectures/16-cls.pdf

[17] D. Ganguli, L. Lovitt, J. Kernion, A. Askell, Y. Bai *et al.* Red teaming language models to reduce harms: methods, scaling behaviors, and lessons learned. arXiv:2209.07858, 2022. https://arxiv.org/abs/2209.07858

[18] OpenAI. GPT-4 technical report. arXiv:2303.08774, 2023. https://arxiv.org/abs/2303.08774

[19] C. Guo, G. Pleiss, Y. Sun y K. Q. Weinberger. On calibration of modern neural networks. En *Proceedings of the 34th International Conference on Machine Learning (ICML)*, PMLR 70, 2017. https://arxiv.org/abs/1706.04599

[20] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu *et al.* Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023. https://arxiv.org/abs/2211.09110

[21] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song y J. Steinhardt. Measuring massive multitask language understanding. En *International Conference on Learning Representations (ICLR)*, 2021. https://arxiv.org/abs/2009.03300

[22] J. Hewitt y P. Liang. Designing and interpreting probes with control tasks. En *Proceedings of EMNLP-IJCNLP*, 2019. https://arxiv.org/abs/1909.03368

[23] J. Hewitt y C. D. Manning. A structural probe for finding syntax in word representations. En *Proceedings of NAACL-HLT*, págs. 4129–4138, 2019. https://aclanthology.org/N19-1419/

[24] E. Hubinger, C. Denison, J. Mu, M. Lambert, M. Tong *et al.* Sleeper agents: training deceptive LLMs that persist through safety training. arXiv:2401.05566, 2024. https://arxiv.org/abs/2401.05566

[25] D. Jurafsky y J. H. Martin. *Speech and Language Processing*, 3.ª ed., borrador del 19 de agosto de 2026. Cap. 7, «Transformers and Pretraining»; cap. 10, «Interpretability». https://web.stanford.edu/~jurafsky/slp3/

[26] S. Kadavath, T. Conerly, A. Askell, T. Henighan, D. Drain *et al.* Language models (mostly) know what they know. arXiv:2207.05221, 2022. https://arxiv.org/abs/2207.05221

[27] A. T. Kalai y S. S. Vempala. Calibrated language models must hallucinate. arXiv:2311.14648, 2023. https://arxiv.org/abs/2311.14648

[28] A. T. Kalai, O. Nachum, S. S. Vempala y E. Zhang. Why language models hallucinate. arXiv:2509.04664, 2025. https://arxiv.org/abs/2509.04664

[29] K. Li, O. Patel, F. Viégas, H. Pfister y M. Wattenberg. Inference-time intervention: eliciting truthful answers from a language model. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2306.03341

[30] T. Lieberum, S. Rajamanoharan, A. Conmy, L. Smith, N. Sonnerat, V. Varma, J. Kramár, A. Dragan, R. Shah y N. Nanda. Gemma Scope: open sparse autoencoders everywhere all at once on Gemma 2. arXiv:2408.05147, 2024. https://arxiv.org/abs/2408.05147

[31] J. Lindsey, W. Gurnee, E. Ameisen, B. Chen, A. Pearce *et al.* On the biology of a large language model. *Transformer Circuits Thread*, Anthropic, 27 de marzo de 2025. https://transformer-circuits.pub/2025/attribution-graphs/biology.html

[32] S. M. Lundberg y S.-I. Lee. A unified approach to interpreting model predictions. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. https://arxiv.org/abs/1705.07874

[33] K. Meng, D. Bau, A. Andonian e Y. Belinkov. Locating and editing factual associations in GPT. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. https://arxiv.org/abs/2202.05262

[34] J. Orloff y J. Bloom. 18.05: Introduction to Probability and Statistics. Massachusetts Institute of Technology, MIT OpenCourseWare, primavera de 2022. Clase 23, «Confidence Intervals for the Mean of Non-normal Data», y sus *in-class problems*. https://ocw.mit.edu/courses/18-05-introduction-to-probability-and-statistics-spring-2022/

[35] N. Nanda, L. Chan, T. Lieberum, J. Smith y J. Steinhardt. Progress measures for grokking via mechanistic interpretability. En *International Conference on Learning Representations (ICLR)*, 2023. https://arxiv.org/abs/2301.05217

[36] nostalgebraist. Interpreting GPT: the logit lens. *LessWrong*, 2020.

[37] C. Olah, N. Cammarata, L. Schubert, G. Goh, M. Petrov y S. Carter. Zoom in: an introduction to circuits. *Distill*, 2020. https://distill.pub/2020/circuits/zoom-in/

[38] C. Olsson, N. Elhage, N. Nanda, N. Joseph, N. DasSarma, T. Henighan *et al.* In-context learning and induction heads. *Transformer Circuits Thread*, Anthropic, 2022. https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html

[39] K. Park, Y. J. Choe y V. Veitch. The linear representation hypothesis and the geometry of large language models. arXiv:2311.03658, 2023. https://arxiv.org/abs/2311.03658

[40] E. Perez, S. Huang, F. Song, T. Cai, R. Ring, J. Aslanides, A. Glaese, N. McAleese y G. Irving. Red teaming language models with language models. En *Proceedings of EMNLP*, 2022. https://arxiv.org/abs/2202.03286

[41] Anthropic. Anthropic's Responsible Scaling Policy. Septiembre de 2023, con revisiones posteriores. https://www.anthropic.com/news/anthropics-responsible-scaling-policy

[42] M. Sundararajan, A. Taly y Q. Yan. Axiomatic attribution for deep networks. En *Proceedings of the 34th International Conference on Machine Learning (ICML)*, PMLR 70, 2017. https://arxiv.org/abs/1703.01365

[43] A. Templeton, T. Conerly, J. Marcus, J. Lindsey, T. Bricken *et al.* Scaling monosemanticity: extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, Anthropic, 2024. https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html

[44] A. M. Turner, L. Thiergart, G. Leech, D. Udell, J. J. Vazquez, U. Mini y M. MacDiarmid. Steering language models with activation engineering. arXiv:2308.10248, 2023. https://arxiv.org/abs/2308.10248

[45] K. Wang, A. Variengien, A. Conmy, B. Shlegeris y J. Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. En *International Conference on Learning Representations (ICLR)*, 2023. https://arxiv.org/abs/2211.00593

[46] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. §3.7, «Weight Decay». https://d2l.ai/

[47] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter y M. Fredrikson. Universal and transferable adversarial attacks on aligned language models. arXiv:2307.15043, 2023. https://arxiv.org/abs/2307.15043